

Segmentación de Vasos Sanguíneos en Imágenes de Retina mediante Redes Neuronales Convolucionales

Alexander Velez

May 16, 2025

Abstract

Este trabajo estudia y evalúa arquitecturas de redes neuronales convolucionales para la segmentación precisa de vasos sanguíneos en imágenes de fondo de ojo de alta resolución. La detección automática y precisa de la red vascular retiniana es fundamental para el diagnóstico temprano de patologías como la retinopatía diabética, la hipertensión arterial y el glaucoma. Se analizan dos arquitecturas principales: FRNet (Full Resolution Network) y RoiNet (también conocida como VesselView), cada una con enfoques distintos para abordar los desafíos específicos que presenta la segmentación de estructuras vasculares finas. Se pone un énfasis particular en el análisis, las modificaciones y los experimentos realizados por el autor sobre la arquitectura RoiNet. Los resultados experimentales derivados de este trabajo, especialmente con la versión modificada y evaluada de RoiNet, muestran un rendimiento prometedor en términos de precisión y eficiencia computacional en comparación con los métodos existentes, especialmente cuando se trabaja con imágenes de alta resolución (2048×2048 píxeles).

Contents

List of Tables

List of Figures

Chapter 1

Introducción

Las enfermedades oculares representan un problema de salud pública significativo a nivel mundial. La detección temprana de patologías como la retinopatía diabética, el glaucoma o la degeneración macular asociada a la edad puede prevenir la pérdida de visión en millones de personas. Las imágenes de fondo de ojo (retinografías) constituyen una herramienta no invasiva fundamental para el diagnóstico de estas enfermedades, permitiendo visualizar la estructura vascular retiniana. La segmentación automática de los vasos sanguíneos en estas imágenes es un paso crucial para el desarrollo de sistemas de diagnóstico asistido por ordenador (CAD), ya que el análisis morfológico de la vasculatura (grosor, tortuosidad, ángulos de ramificación, etc.) proporciona biomarcadores importantes.

Sin embargo, esta tarea presenta desafíos significativos debido a:

- La estructura fina y ramificada de los vasos sanguíneos, especialmente los capilares más pequeños.
- La variabilidad en el contraste entre los vasos y el fondo retiniano, a menudo afectada por la pigmentación o la presencia de patologías.
- La presencia de lesiones (exudados, hemorragias) y artefactos (iluminación no uniforme, pestañas) que pueden confundirse con estructuras vasculares.
- La necesidad de mantener la conectividad y continuidad de la red vascular para un análisis topológico correcto.

Las técnicas de procesamiento de imágenes han evolucionado considerablemente. Inicialmente, se emplearon métodos basados en filtros, detección de bordes y umbralización, que si bien sentaron las bases, a menudo requerían ajustes manuales y eran sensibles al ruido y a las variaciones en las imágenes. Posteriormente, los enfoques de aprendizaje automático clásico intentaron mejorar la robustez, pero dependían de una ingeniería de características manual. En los últimos años, las redes neuronales convolucionales (CNN) han revolucionado el campo de la segmentación de imágenes médicas, ofreciendo resultados prometedores en la segmentación vascular retiniana gracias a su capacidad de aprender jerarquías de características directamente de los datos.

Este trabajo se centra en el estudio, la adaptación y la evaluación de arquitecturas CNN especializadas para la segmentación de vasos sanguíneos en imágenes de retina de alta resolución, con énfasis en la preservación de estructuras finas, la eficiencia computacional y, fundamentalmente, en las contribuciones y experimentos realizados por el autor sobre la arquitectura RoiNet.

Chapter 2

Estado del arte

La segmentación de vasos sanguíneos en imágenes de retina ha sido objeto de estudio durante décadas, evolucionando desde métodos basados en técnicas de procesamiento de imágenes clásicas hasta los actuales enfoques basados en aprendizaje profundo.

2.1 Métodos tradicionales

Los primeros enfoques para la segmentación vascular retiniana se basaban principalmente en:

- **Métodos de umbralización:** Utilizan diferentes técnicas de umbralización (global, local, adaptativa) para separar los vasos del fondo basándose en la intensidad de los píxeles. Aunque simples, son muy sensibles a variaciones de contraste e iluminación.
- **Métodos basados en bordes:** Aplican operadores de detección de bordes como Sobel, Prewitt, Roberts o Canny para identificar los límites de los vasos. Suelen generar bordes fragmentados y requieren post-procesamiento.
- **Métodos morfológicos:** Utilizan operaciones morfológicas como la apertura, el cierre, el top-hat o la reconstrucción morfológica para extraer estructuras vasculares. Son útiles para eliminar ruido o realzar ciertas formas, pero su efectividad depende mucho de la elección del elemento estructurante.
- **Métodos basados en coincidencia de patrones (Matched Filtering):** Emplean filtros adaptados (kernels 2D que modelan el perfil de intensidad de un vaso) para detectar estructuras tubulares. El filtro de Gabor es un ejemplo común. Aunque efectivos para vasos de cierto calibre, pueden fallar en vasos muy finos o en puntos de cruce.
- **Seguimiento de vasos (Vessel Tracking):** Partiendo de puntos semilla, intentan seguir el curso de los vasos. Pueden ser robustos pero dependen de una buena inicialización y pueden tener problemas en bifurcaciones o cruces.

Estos métodos, aunque computacionalmente eficientes en su mayoría, presentan limitaciones significativas en términos de precisión y robustez, especialmente en presencia de patologías, artefactos, o variaciones anatómicas. Muchos de estos enfoques más antiguos ya no se utilizan como métodos principales, aunque algunos de sus principios pueden encontrarse en etapas de pre o post-procesamiento de sistemas más modernos.

2.2 Métodos basados en aprendizaje automático

Con el avance de las técnicas de aprendizaje automático, surgieron enfoques que combinan la extracción de características y clasificadores supervisados:

- **Métodos basados en características:** Extraen un conjunto de características para cada píxel (o región) como textura, intensidad, información de color, respuesta a filtros, y geometría local. Luego, utilizan clasificadores como máquinas de vectores de soporte (SVM), k-vecinos más cercanos (k-NN), AdaBoost o Random Forest para la segmentación.
- **Métodos basados en modelos:** Utilizan modelos deformables (contornos activos o "snakes") o modelos estadísticos de forma para ajustarse a la estructura vascular.

Estos métodos mejoraron la precisión de la segmentación respecto a los tradicionales, pero seguían dependiendo en gran medida de la calidad y relevancia de las características extraídas manualmente, un proceso que requiere conocimiento experto y puede no generalizar bien.

2.3 Métodos basados en aprendizaje profundo

El surgimiento del aprendizaje profundo, y en particular las Redes Neuronales Convolucionales (CNN), ha transformado radicalmente el campo de la segmentación de imágenes médicas:

- **U-Net:** Propuesta por Ronneberger et al. en 2015, esta arquitectura encoder-decoder con conexiones de salto (skip connections) se ha convertido en un estándar de facto para la segmentación de imágenes biomédicas, gracias a su capacidad para capturar contexto y localizar con precisión.
- **Arquitecturas basadas en ResNet:** Incorporan conexiones residuales (ResNet) para facilitar el entrenamiento de redes más profundas y mejorar la propagación del gradiente, permitiendo modelos con mayor capacidad representacional.
- **Redes con atención:** Integran mecanismos de atención (espacial, por canal, o combinados) para que el modelo aprenda a enfocarse en las regiones o características más relevantes de la imagen, mejorando la eficiencia y la precisión.
- **Otras variantes:** Han surgido múltiples variantes y mejoras sobre estas ideas base, como U-Net++, Attention U-Net, DenseNet, etc., cada una buscando abordar limitaciones específicas.

A pesar de los avances significativos, estos métodos aún enfrentan desafíos cuando se trata de segmentar estructuras vasculares finas en imágenes de alta resolución, principalmente debido a:

1. La pérdida de información detallada durante las operaciones de reducción de resolución (pooling) en arquitecturas encoder-decoder.
2. El alto coste computacional y de memoria al procesar imágenes de gran tamaño de forma directa.
3. La dificultad para mantener la conectividad de estructuras finas y elongadas, y para distinguir vasos de artefactos o lesiones similares.

Este trabajo se enfoca en el estudio y la adaptación de arquitecturas CNN, con especial atención a RoiNet (VesselView) y las contribuciones del autor sobre ella, para abordar estos desafíos, utilizando funciones de pérdida y estrategias de entrenamiento adecuadas a la morfología vascular y a las particularidades del TFG.

2.3.1 Datasets disponibles

En el ámbito de la segmentación de imágenes de retina, se han utilizado diversos conjuntos de datos para entrenar y evaluar modelos. Entre los más destacados se encuentran:

- **DRIVE**: Un conjunto de datos ampliamente utilizado que contiene imágenes de retina anotadas para la segmentación de vasos.
- **STARE**: Otro conjunto de datos popular que proporciona imágenes de retina con anotaciones detalladas.
- **CHASE_DB1**: Conjunto de datos que ofrece imágenes de retina de alta calidad con anotaciones de vasos.

En este trabajo, utilizamos el dataset **FIVES** debido a su alta calidad y resolución (2048x2048 píxeles), lo que permite una evaluación precisa de las arquitecturas propuestas. También se consideró el uso de versiones adaptadas como FIVES512 para experimentación en entornos con recursos limitados.

Chapter 3

Objetivos del Trabajo Fin de Grado

Conforme a las directrices del proyecto de TFG y las correcciones recibidas, los objetivos de este trabajo se centran en las aportaciones y el análisis realizado por el autor, en lugar de presentar una evolución lineal de arquitecturas no desarrolladas íntegramente en este marco. Los objetivos específicos son:

1. **Analizar y Describir Arquitecturas Existentes:** Estudiar y presentar las características fundamentales de dos arquitecturas relevantes para la segmentación vascular: FRNet, por su enfoque de resolución completa, y RoiNet (VesselView), una arquitectura tipo U-Net que sirve como base para el trabajo experimental del TFG.
2. **Adaptar y Optimizar la Arquitectura RoiNet (VesselView):** Focalizar los esfuerzos en la arquitectura RoiNet, realizando las adaptaciones y optimizaciones necesarias para la segmentación de vasos sanguíneos en imágenes de retina de alta resolución. Esta versión modificada y experimentada por el autor es la que se ha denominado informalmente "SantosNet" en discusiones con el tutor.
3. **Detallar y Justificar las Aportaciones Propias sobre RoiNet:** Documentar exhaustivamente las contribuciones realizadas por el autor sobre la arquitectura RoiNet/VesselView. Esto incluye:
 - Desarrollo y adaptación de scripts para la ejecución y experimentación en plataformas de computación de alto rendimiento (Cesga).
 - Modificaciones sistemáticas en la configuración del entrenamiento, explorando diversos hiperparámetros y estrategias.
 - Análisis y documentación de los desafíos prácticos encontrados durante la experimentación (e.g., gestión de recursos en Cesga, necesidad de trabajar con FIVES512) y las soluciones implementadas.
 - Implementación, integración y evaluación de diferentes funciones de pérdida (e.g., SoftCLDice) para mejorar la calidad de la segmentación.
 - Diseño y ejecución de estudios de ablación para comprender el impacto de componentes específicos de la red RoiNet (e.g., variaciones en bloques, métodos de fusión de skip connections).
 - Experimentación con variaciones arquitectónicas dentro de RoiNet.
 - Inicio del desarrollo y prueba de una nueva función de pérdida adaptada a los objetivos del proyecto. [PENDIENTE: Detallar si esta función de pérdida se

concretó]

4. **Realizar un Análisis Exhaustivo de Resultados:** Evaluar y comparar el rendimiento de las arquitecturas (FRNet como referencia y RoiNet con las modificaciones del autor) mediante métricas cuantitativas y análisis cualitativo. Interpretar los resultados para entender las fortalezas y debilidades de cada enfoque y de los componentes evaluados.
5. **Documentar el Proceso de Desarrollo:** Registrar el proceso de refactorización del código, la integración de nuevas funcionalidades y las dificultades técnicas superadas.
6. **Elaborar Conclusiones y Proponer Trabajo Futuro:** Sintetizar los hallazgos del TFG, valorar el aprendizaje obtenido y proponer líneas de investigación futuras basadas en el trabajo realizado con RoiNet/VesselView ("SantosNet").

Este TFG no busca narrar una transición de FRNet a RoiNet como un desarrollo propio, sino centrarse en el trabajo efectivo realizado por el autor desde el inicio del proyecto, utilizando RoiNet/VesselView como el principal campo de experimentación y aportación.

Chapter 4

Metodología

4.1 Arquitectura FRNet (Full Resolution Network)

FRNet es una arquitectura neuronal diseñada específicamente para la segmentación de imágenes médicas, con un enfoque en el procesamiento de características manteniendo la resolución espacial completa durante todo el flujo de datos. Esta arquitectura se estudia en este trabajo como un ejemplo de enfoque full-resolution para abordar:

1. **El problema de preservación de estructuras finas:** Los vasos sanguíneos son estructuras delgadas que podrían perderse con reducciones de resolución.
2. **La necesidad de procesamiento eficiente:** Capacidad para trabajar con imágenes de alta resolución manteniendo un uso razonable de recursos.

4.1.1 Fundamentos y diseño conceptual

FRNet se origina como una adaptación inspirada en una arquitectura Full-Resolution, modificada específicamente para tareas de segmentación vascular. La idea central es mantener la resolución espacial completa, evitando las operaciones de downsampling y upsampling típicas de las arquitecturas encoder-decoder. Esta filosofía de diseño difiere fundamentalmente de las arquitecturas U-Net tradicionales, que reducen la resolución para capturar contexto y luego la recuperan para la segmentación detallada.

FRNet es una de las arquitecturas de referencia estudiadas en este trabajo por su enfoque en el procesamiento a resolución completa. La preservación de la resolución completa en todo momento permite que la red mantenga la información espacial detallada necesaria para segmentar estructuras vasculares finas, que son críticas en el diagnóstico de patologías retinianas.

4.1.2 Implementación técnica

La arquitectura FRNet se caracteriza por:

```
1 class FRNet(nn.Module):
2     def __init__(self, ch_in, ch_out, ls_mid_ch=([32]*6), out_k_size=11,
3         k_size=3,
4         cls_init_block=ResidualBlock, cls_conv_block=
5         ResidualBlock):
6         super().__init__()
7         self.dict_module = nn.ModuleDict()
8         ch = ch_in
```

```

8      # Bloque inicial
9      self.dict_module.add_module(f'conv0', cls_init_block(ch,
10     ls_mid_ch[0], k_size=k_size))
11     ch = ls_mid_ch[0]
12
13     # Bloques intermedios
14     for i in range(1, len(ls_mid_ch)):
15         ch1 = ls_mid_ch[i-1]
16         ch2 = ls_mid_ch[i]
17         self.dict_module.add_module(f'conv{i}', cls_conv_block(ch1,
18         ch2, k_size=k_size))
19
20     # Bloque final
21     ch1 = ls_mid_ch[-1]
22     self.dict_module.add_module("final", nn.Sequential(
23         nn.Conv2d(ch1, ch_out*4, out_k_size, padding=out_k_size//2,
24         bias=False),
25         nn.Sigmoid()
26     ))

```

Listing 4.1: Implementación de FRNet

- **Estructura secuencial de bloques residuales:** Cadena de bloques convolucionales que mantienen la resolución constante a lo largo de toda la red.
- **Kernel final amplio:** Utiliza un kernel de salida de 11×11 para capturar mayor contexto en la decisión final, lo que permite integrar información de un área más amplia para cada píxel de salida.
- **Sistema modular:** Permite intercambiar diferentes tipos de bloques convolucionales según las necesidades, lo que facilita la experimentación con variantes arquitectónicas.
- **Profundidad ajustable:** La lista `ls_mid_ch` define el número de canales en cada capa intermedia, permitiendo ajustar la profundidad y capacidad de la red.

En el método `forward`, la red procesa secuencialmente la entrada:

```

1 def forward(self, x):
2     for i in range(len(self.ls_mid_ch)):
3         conv = self.dict_module[f'conv{i}']
4         x = conv(x)
5
6     x = self.dict_module['final'](x)
7     x = torch.max(x, dim=1, keepdim=True)[0]
8     return x

```

Listing 4.2: Método forward de FRNet

Este diseño de flujo de datos lineal facilita la propagación de gradientes durante el entrenamiento y mantiene un uso de memoria predecible, proporcional a la resolución de entrada.

4.1.3 ResidualBlock como unidad básica

```

1 class ResidualBlock(ConvBlock):

```

```

2     def init(self, in_channels, out_channels, stride, k_size, dilation,
layer_num=None):
3         p = k_size//2 * dilation
4         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=
k_size,
5                                 stride=stride, padding=p, bias=False,
dilation=dilation)
6         self.bn1 = nn.BatchNorm2d(out_channels)
7         self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=
k_size,
8                                 stride=1, padding=p, bias=False, dilation
=dilation)
9         self.bn2 = nn.BatchNorm2d(out_channels)
10
11        self.shortcut = nn.Sequential()
12        if stride != 1 or in_channels != out_channels:
13            self.shortcut = nn.Sequential(
14                nn.Conv2d(in_channels, out_channels, kernel_size=1,
15                            stride=stride, bias=False),
16                nn.BatchNorm2d(out_channels)
17            )

```

Listing 4.3: Implementación de ResidualBlock

El bloque residual implementa:

- **Doble capa convolucional con normalización por lotes:** Permite un procesamiento más profundo de las características mientras mantiene la estabilidad del entrenamiento.
- **Conexión residual (shortcut):** Facilita el flujo de gradientes durante el entrenamiento, mitigando el problema del desvanecimiento del gradiente y permitiendo entrenar redes más profundas.
- **Soporte para dilatación:** Permite aumentar el campo receptivo sin incrementar el número de parámetros o reducir la resolución espacial, lo que es crucial para capturar contexto en estructuras vasculares.
- **Padding adaptativo:** Se ajusta automáticamente según el tamaño del kernel y la dilatación, garantizando que la resolución espacial se mantenga constante.

4.1.4 Capa final especializada

```

1 self.dict_module.add_module(f"final", nn.Sequential(
2     nn.Conv2d(ch1, ch_out*4, out_k_size, padding=out_k_size//2, bias=
False),
3     nn.Sigmoid()
4 ))

```

Listing 4.4: Capa final especializada de FRNet

La capa final incorpora:

- **Kernel grande (11×11):** Proporciona un contexto local amplio para la decisión de segmentación final, permitiendo considerar la estructura vascular circundante.

- **Generación de múltiples mapas (`ch_out*4`):** Produce cuatro veces más canales que los necesarios para la salida final, lo que añade robustez a la predicción.
- **Activación sigmoide:** Normaliza las salidas al rango $[0,1]$, adecuado para la segmentación binaria.
- **Selección del máximo:** La operación `torch.max(x, dim=1, keepdim=True)[0]` selecciona el valor máximo entre los canales generados, implementando un mecanismo de "voto" entre múltiples candidatos de segmentación.

4.1.5 Efectos esperados y consideraciones

Con esta arquitectura, se podrían anticipar los siguientes efectos:

1. **Preservación de detalles finos:**

- Mejor conservación de vasos capilares delgados al no perder resolución espacial
- Potencial mejora en la continuidad de estructuras vasculares

2. **Comportamiento en entrenamiento:**

- Posible convergencia más rápida por la simplicidad del flujo de datos
- Aprovechamiento de batch sizes mayores al requerir menos memoria por imagen

3. **Limitaciones potenciales:**

- Campo receptivo limitado que podría afectar la captura de contexto global
- Posible dificultad para capturar relaciones de largo alcance entre estructuras vasculares

4.2 Arquitectura RoiNet (VesselView)

RoiNet (también conocida como VesselView, y base para las modificaciones y experimentación realizadas en este TFG) es otra arquitectura clave analizada, que adopta un enfoque inspirado en U-Net para abordar algunas limitaciones identificadas en el diseño full-resolution. Esta adaptación busca:

1. **Mejorar la captura de contexto global:** Mediante procesamiento multiescala con downsampling/upsampling
2. **Mantener la capacidad de preservar detalles:** A través de skip connections estratégicas
3. **Balancear resolución y contexto:** Combinando características de diferentes niveles de resolución

4.2.1 Fundamentos y diseño conceptual

RoiNet, también conocida como VesselView en la publicación asociada a este trabajo [PENDIENTE: Citar publicación si aplica], representa un enfoque diferente a FRNet. Mientras que FRNet mantiene una resolución constante, RoiNet adopta un paradigma encoder-decoder inspirado en U-Net, pero con modificaciones sustanciales:

1. **Bloques residuales de doble convolución:** Utiliza bloques residuales con convoluciones de 9×9 , lo que aumenta el campo receptivo y ayuda a preservar los detalles finos de los vasos.
2. **Cuello de botella profundo:** Fortalece la extracción de características semánticas de alto nivel, vitales para segmentar vasos delgados.
3. **Conexiones de salto refinadas:** A diferencia de la concatenación directa de U-Net,

RoiNet utiliza conexiones de salto refinadas con convoluciones adicionales para mejorar la integración de características locales y globales.

Esta arquitectura busca combinar lo mejor de ambos mundos: la capacidad de FRNet para preservar detalles finos y la habilidad de U-Net para capturar contexto global.

4.2.2 Implementación técnica

La arquitectura de RoiNet se basa en un marco de codificador-decodificador enriquecido con conexiones de salto estratégicas y un cuello de botella profundo:

```

1 class RoiNet(nn.Module):
2     def __init__(self, ch_in, ch_out, ls_mid_ch=[32, 64, 128, 128, 64,
3         32],
4         k_size=9, cls_init_block=ResidualBlock, cls_conv_block=
5         ResidualBlock):
6         super().__init__()
7         self.dict_module = nn.ModuleDict()
8         ch = ch_in # current channel count
9
10        # ----- Encoder -----
11        # Block 0: Full resolution features.
12        self.dict_module.add_module("conv0", cls_init_block(ch,
13            ls_mid_ch[0], k_size=k_size, layer_num=0))
14        ch = ls_mid_ch[0] # 32
15
16        # Block 1: Downsample once.
17        self.dict_module.add_module("conv1", cls_init_block(ch,
18            ls_mid_ch[1], k_size=k_size, layer_num=1))
19        ch = ls_mid_ch[1] # 64
20        # Downsample & double channels.
21        self.dict_module.add_module("pool1", nn.Sequential(
22            nn.MaxPool2d(kernel_size=2, stride=2),
23            nn.Conv2d(ch, ch * 2, kernel_size=1)
24        ))
25        ch = ch * 2 # becomes 128
26        # Skip connection "skip1"
27
28        # Block 2: Further encoding.
29        self.dict_module.add_module("conv2", cls_init_block(ch,
30            ls_mid_ch[2], k_size=k_size, layer_num=2))
31        ch = ls_mid_ch[2] # 128
32        # Downsample & double channels.
33        self.dict_module.add_module("pool2", nn.Sequential(
34            nn.MaxPool2d(kernel_size=2, stride=2),
35            nn.Conv2d(ch, ch * 2, kernel_size=1)
36        ))
37        ch = ch * 2 # becomes 256
38        # Skip connection "skip2"

```

Listing 4.5: Implementación de RoiNet (Encoder)

- **Camino del codificador:** Extrae características jerárquicas de las imágenes de fondo de

ojo, utilizando convoluciones de bloque residual con kernels de 9×9 para capturar contextos espaciales más amplios. La resolución se reduce progresivamente mientras se aumenta el número de canales.

- **Cuello de botella profundo:** Presenta dos bloques residuales con 256 canales, fortaleciendo la extracción de características semánticas de alto nivel:

```

1 # ----- Bottleneck (Deepened) -----
2 self.dict_module.add_module("bottle1", cls_conv_block(ch, ch, k_size=
   k_size, layer_num="bottle1"))
3 self.dict_module.add_module("bottle2", cls_conv_block(ch, ch, k_size=
   k_size, layer_num="bottle2"))
4 # Merge skip2 (from encoder) with the deepened bottleneck output.
5 self.dict_module.add_module("merge2", nn.Sequential(
6     nn.Conv2d(ch * 2, ch, kernel_size=3, padding=1, bias=False),
7     nn.BatchNorm2d(ch),
8     nn.ReLU(inplace=True)
9 ))

```

Listing 4.6: Implementación de RoiNet (Bottleneck)

- **Camino del decodificador:** Reconstruye la resolución espacial y refina las predicciones de segmentación, utilizando convoluciones transpuestas y conexiones de salto refinadas:

```

1 # ----- Decoder -----
2 # Block 3: Upsample from bottleneck.
3 self.dict_module.add_module("conv3", cls_init_block(ch, ls_mid_ch[3],
   k_size=k_size, layer_num=3))
4 ch = ls_mid_ch[3] # 128
5 self.dict_module.add_module("up3", nn.Sequential(
6     nn.ConvTranspose2d(ch, ch // 2, kernel_size=2, stride=2)
7 ))
8 ch = ch // 2 # becomes 64
9 # Merge with skip connection from Block 1
10 self.dict_module.add_module("merge3", nn.Sequential(
11     nn.Conv2d((ls_mid_ch[3] // 2) + (ls_mid_ch[1] * 2), ls_mid_ch[1],
12     kernel_size=3, padding=1, bias=False),
13     nn.BatchNorm2d(ls_mid_ch[1]),
14     nn.ReLU(inplace=True)
15 ))

```

Listing 4.7: Implementación de RoiNet (Decoder)

4.2.3 Flujo de procesamiento y skip connections

El flujo de procesamiento en RoiNet es considerablemente más complejo que en FRNet, con información fluyendo tanto verticalmente (a través de las capas del encoder y decoder) como horizontalmente (a través de las skip connections):

```

1 def forward(self, x):
2     # Encoder
3     out0 = self.dict_module["conv0"](x) # (B, 32, H, W) ->
   skip0

```

```

4     out1 = self.dict_module["conv1"](out0)           # (B, 64, H, W)
5     out1 = self.dict_module["pool1"](out1)           # (B, 128, H/2, W/2)
-> skip1
6     skip1 = out1
7
8     out2 = self.dict_module["conv2"](out1)           # (B, 128, H/2, W/2)
9     out2 = self.dict_module["pool2"](out2)           # (B, 256, H/4, W/4)
-> skip2
10    skip2 = out2
11
12    # Bottleneck (deepened)
13    bottle1 = self.dict_module["bottle1"](out2)       # (B, 256, H/4, W/4)
14    bottle2 = self.dict_module["bottle2"](bottle1)    # (B, 256, H/4, W/4)
15    # Merge the original skip2 with the deepened features.
16    bottle_cat = torch.cat([bottle2, skip2], dim=1)   # (B, 512, H/4,
W/4)
17    bottle_out = self.dict_module["merge2"](bottle_cat) # (B, 256, H/4,
W/4)
18
19    # Decoder
20    out3 = self.dict_module["conv3"](bottle_out)     # (B, 128, H/4, W/4)
21    out3 = self.dict_module["up3"](out3)              # (B, 64, H/2, W/2)
22    # Merge with skip1 (from pool1)
23    out3 = torch.cat([out3, skip1], dim=1)           # (B, 64+128=192, H/2,
W/2)
24    out3 = self.dict_module["merge3"](out3)          # (B, 64, H/2, W/2)

```

Listing 4.8: Método forward de RoiNet

Las skip connections en RoiNet tienen varias características distintivas:

1. **Conexiones refinadas:** A diferencia de U-Net, donde las skip connections son concatenaciones directas, RoiNet refina estas conexiones mediante bloques convolucionales adicionales (merge2, merge3, etc.).
2. **Integración con bottleneck:** La primera skip connection se integra directamente en el bottleneck, permitiendo que las características de alta resolución influyan en la extracción de características semánticas profundas.
3. **Fusión adaptativa:** Las operaciones de fusión ajustan el número de canales para mantener un crecimiento controlado de las características a medida que se reconstruye la resolución.

4.2.4 Comparativa técnica FRNet vs RoiNet

4.2.5 Innovaciones arquitectónicas clave en RoiNet

1. **Kernels de gran tamaño (9×9):** A diferencia de las arquitecturas U-Net tradicionales que utilizan kernels de 3×3, RoiNet emplea kernels de 9×9 en sus bloques residuales. Esto aumenta significativamente el campo receptivo de cada capa, permitiendo capturar patrones vasculares más amplios sin necesidad de apilar tantas capas.
2. **Bottleneck profundo:** El cuello de botella en RoiNet no es simplemente un punto de transición entre el encoder y el decoder, sino una región profunda con múltiples bloques residuales que procesan intensivamente las características en su resolución más baja. Esto

Table 4.1: Comparativa técnica entre FRNet y RoiNet

Aspecto Técnico	FRNet	RoiNet
Patrón arquitectónico	Lineal, sin cambios de resolución	Encoder-decoder con múltiples saltos
Gestión de resolución	Constante durante todo el procesamiento	Reducción y recuperación de resolución
Estrategia de contexto	Acumulativo a través de capas secuenciales	Multiresolución con caminos paralelos
Transferencia de información	Directa a través de la cadena de bloques	A través de skip connections
Estrategia de salida	Múltiples candidatos con selección de máximo	Proyección directa a canal de salida
Requisitos de memoria	Proporcional a resolución de entrada	Variable según niveles de resolución
Tamaño de kernel	Típicamente 3×3	Ampliado a 9×9 para multiresolución
Bottleneck	No aplicable	Profundo con múltiples saltos

fortalece la capacidad de la red para extraer características semánticas de alto nivel.

3. **Skip connections refinadas con convoluciones 1×1 :** Las conexiones de salto en RoiNet no son simples concatenaciones como en U-Net. Incluyen convoluciones 1×1 que actúan como "puertas de atención" implícitas, permitiendo a la red aprender qué características de alta resolución son más relevantes para cada etapa de reconstrucción.
4. **Fusión adaptativa de características:** Los módulos de fusión (`merge2`, `merge3`, etc.) no solo concatenan características, sino que las refinan mediante convoluciones 3×3 seguidas de normalización por lotes y activación ReLU. Esto permite una integración más suave de las características locales y globales.
5. **Estrategia de pooling seguida de proyección:** En lugar de utilizar convoluciones con stride para reducir la resolución, RoiNet emplea max pooling seguido de una convolución 1×1 . Esto separa conceptualmente la tarea de reducción espacial de la transformación de características, potencialmente mejorando la estabilidad del entrenamiento.

4.2.6 Aportaciones y Modificaciones del Autor sobre RoiNet

En el marco de este TFG, la arquitectura RoiNet (VesselView) sirvió de base para una serie de modificaciones, experimentos y desarrollos propios, conduciendo a una versión adaptada y optimizada. Las principales aportaciones realizadas por el autor, que constituyen el núcleo de este trabajo, incluyen:

- **Adaptación para Computación de Alto Rendimiento (Cesga):**
 - Se crearon y ajustaron scripts específicos para la ejecución eficiente de los procesos de entrenamiento y evaluación en la infraestructura del Centro de Supercomputación de Galicia (Cesga).
 - Esto implicó la gestión de módulos de software, la configuración de trabajos para el sistema de colas (Slurm), la optimización del uso de recursos (CPU, GPU, memoria) y la adaptación a las particularidades del entorno del Cesga.
 - [PENDIENTE: Incluir fragmento de código relevante o descripción más detallada del script de Cesga, por ejemplo, cómo se gestionaban los datasets o los checkpoints]
- **Configuración y Optimización del Entrenamiento:**
 - Se llevó a cabo una exploración sistemática de diversos hiperparámetros y configuraciones de entrenamiento para RoiNet. Esto incluyó la experimentación con diferentes tasas de aprendizaje (learning rates), tamaños de lote (batch sizes), funciones de optimización

(e.g., Adam, SGD), y esquemas de regularización.

- [PENDIENTE: Detallar configuraciones específicas probadas y cuáles ofrecieron mejores resultados. Se puede incluir una tabla resumen si es pertinente.]

- **Gestión de Desafíos Experimentales y Soluciones:**

- Durante la fase experimental en el Cesga, se afrontaron desafíos significativos, como la saturación de recursos computacionales (memoria GPU, tiempo de ejecución en colas).
- Se investigaron e implementaron soluciones alternativas para mitigar estos problemas. Un ejemplo notable fue la adaptación y uso del dataset FIVES512 (una versión de FIVES con imágenes redimensionadas a 512x512 píxeles) para permitir una experimentación más ágil y la prueba de un mayor número de configuraciones en nodos locales o con recursos más limitados, antes de escalar a la resolución completa.

- **Mejoras y Pruebas en la Arquitectura y Funciones de Pérdida:**

- **Función de Pérdida SoftCLDice:** Se integró y evaluó la función de pérdida SoftCLDice (descrita en la sección 4.3.1) en el entrenamiento de RoiNet. El objetivo era mejorar la conectividad topológica de las segmentaciones vasculares, un aspecto crucial para la calidad de la segmentación. [PENDIENTE: Referenciar resultados específicos de esta evaluación en la sección de Resultados]
 - **Estudio de Ablación:** Se diseñaron y ejecutaron estudios de ablación para analizar el impacto de componentes específicos de la arquitectura RoiNet. Esto incluyó la evaluación de:
 - * Diferentes métodos de fusión de características en las skip connections (e.g., concatenación simple vs. convoluciones adicionales para refinar la fusión).
 - * El impacto del número de bloques residuales en diferentes partes de la red, especialmente en el bottleneck.
 - * (Ver sección 5.2.2 para detalles de estos estudios). [PENDIENTE: Asegurar que la sección 5.2.2 detalle estas configuraciones del estudio de ablación]
 - **Variaciones Arquitectónicas:** Se probaron modificaciones estructurales en RoiNet, como variar el número de bloques convolucionales en las etapas del encoder y decoder, o ajustar la profundidad del bottleneck. [PENDIENTE: Especificar qué variaciones concretas de bloques se probaron, dónde, y cuáles fueron sus efectos observados]
 - **Desarrollo de Nueva Función de Pérdida:** Se inició el diseño y las pruebas preliminares de una nueva función de pérdida. El objetivo de esta función era abordar de forma más específica ciertos desafíos de la segmentación vascular, como el desequilibrio entre vasos finos y gruesos o la penalización de discontinuidades. [PENDIENTE: Describir brevemente la formulación o el concepto de esta nueva función de pérdida y el estado de su desarrollo/evaluación al finalizar el TFG]
- **Integración y Refactorización de Código:**
- Se realizó un trabajo considerable en la integración de todas las nuevas funciones, módulos (como las funciones de pérdida adicionales) y modificaciones dentro de la base de código existente del proyecto.

- Paralelamente, se llevaron a cabo tareas de refactorización del código para mejorar su estructura, modularidad, legibilidad y mantenibilidad, facilitando la experimentación y la futura extensibilidad.
- **Documentación de Dificultades Técnicas:**
 - A lo largo del TFG, se documentaron las diversas dificultades técnicas encontradas, tanto a nivel de implementación (e.g., compatibilidad de librerías, debugging de modelos profundos) como de experimentación (e.g., gestión de grandes volúmenes de datos, reproducibilidad). Se registraron también las soluciones y estrategias adoptadas para superar estos obstáculos.

Estas aportaciones constituyen el cuerpo principal del trabajo desarrollado en este TFG, centrándose en la aplicación práctica, evaluación y mejora incremental de una arquitectura de aprendizaje profundo para un problema biomédico complejo.

4.3 Funciones de pérdida especializadas

Para abordar los desafíos específicos de la segmentación vascular, se han implementado varias funciones de pérdida especializadas:

4.3.1 SoftCLDiceLoss

Esta función de pérdida combina la pérdida de Dice tradicional con un término que promueve la alineación de los esqueletos (centerlines) de las estructuras vasculares predichas y ground truth. Esto favorece la conectividad y continuidad de los vasos segmentados.

4.3.2 ConexLoss

Diseñada específicamente para penalizar desconexiones en la segmentación vascular. Calcula el gradiente de la predicción y penaliza transiciones bruscas en regiones donde se espera que haya vasos, promoviendo así segmentaciones más suaves y continuas.

4.3.3 DistanceWeightedBCELoss

Una variante de la Binary Cross Entropy ponderada por la distancia euclídea al píxel de vaso más cercano. Los píxeles cercanos a los vasos reciben mayor peso, lo que ayuda a obtener bordes más precisos.

4.3.4 VesselHaloLoss

Combina BCE estándar con un término adicional que penaliza un halo (una banda estrecha alrededor de los bordes del vaso) para conseguir contornos más nítidos.

4.3.5 HaloCLDiceLoss

Fusiona VesselHaloLoss con SoftCLDiceLoss para promover simultáneamente la conectividad y los bordes precisos.

4.4 Conjunto de datos y preprocesamiento

4.4.1 Dataset FIVES

El sistema de entrenamiento está diseñado para trabajar con el dataset FIVES en varios formatos:

- **Alta resolución (2048×2048):** Para evaluar el rendimiento en condiciones óptimas

- **Resoluciones reducidas:** Para comparativas con métodos tradicionales

4.4.2 Preprocesamiento de datos

El preprocesamiento incluye:

- Normalización de intensidades
- Padding para asegurar dimensiones múltiplos de 32
- Extracción del canal verde para maximizar el contraste vascular

4.4.3 Estrategias de aumentación de datos

Para aumentar la robustez del entrenamiento, se implementan:

1. **Transformaciones geométricas:** Rotaciones, flips, escalado
2. **Deformaciones elásticas:** Para simular variabilidad vascular
3. **Modificaciones de intensidad:** Variaciones de brillo y contraste

Chapter 5

Experimentos y Resultados

5.1 Configuración experimental

5.1.1 Entorno de implementación

Los experimentos se realizaron utilizando PyTorch como framework de aprendizaje profundo, en un entorno con las siguientes características:

- GPU: NVIDIA RTX 3090 con 24GB de memoria
- Optimizador: Adam con learning rate adaptativo
- Batch size: Adaptado según la resolución de las imágenes y la arquitectura

5.1.2 Métricas de evaluación

Para evaluar el rendimiento de los modelos se utilizaron las siguientes métricas:

- **Dice Coefficient:** Mide la superposición entre la segmentación predicha y el ground truth
- **Precisión y Recall:** Evalúan la exactitud de la detección de píxeles vasculares
- **F1-Score:** Media armónica de precisión y recall
- **AUC-ROC:** Área bajo la curva ROC, evalúa la capacidad discriminativa del modelo

5.2 Comparativa de arquitecturas

5.2.1 FRNet vs. RoiNet (versión del autor)

Se realizó una comparativa exhaustiva entre la arquitectura FRNet (como ejemplo de enfoque full-resolution) y la arquitectura RoiNet con las adaptaciones y optimizaciones desarrolladas en este TFG. La comparación se centró en:

- Precisión de segmentación (Dice score)
- Eficiencia computacional (tiempo de inferencia y uso de memoria)
- Capacidad para preservar estructuras finas
- Comportamiento en diferentes resoluciones

Los resultados mostraron que:

- FRNet destaca en ciertos aspectos relacionados con la preservación de detalles finos gracias a su procesamiento a resolución completa, y puede ser computacionalmente eficiente para ciertas configuraciones.
- RoiNet, con las modificaciones y el entrenamiento optimizado en este TFG, ofrece un

rendimiento global robusto, mostrando un buen equilibrio entre la captura de contexto y la preservación de detalles. [PENDIENTE: Ser más específico aquí cuando se tengan los resultados finales, e.g., "La RoiNet modificada alcanzó un Dice de X, superando a FRNet en Y bajo tales condiciones..."]

- Ambas arquitecturas, en sus respectivas configuraciones óptimas encontradas, pueden superar a los métodos tradicionales y a CNN genéricas no especializadas para esta tarea.

[PENDIENTE: Insertar tabla comparativa de métricas entre FRNet y la versión final de RoiNet del TFG]

[PENDIENTE: Insertar ejemplos visuales comparativos de segmentaciones de FRNet y RoiNet del TFG]

5.2.2 Estudio de ablación (sobre RoiNet)

Se realizaron estudios de ablación sobre la arquitectura RoiNet modificada para evaluar la contribución de diferentes componentes y decisiones de diseño implementadas por el autor:

- **Impacto del número de bloques en el bottleneck de RoiNet:** Se evaluaron configuraciones con diferente profundidad en el bottleneck para determinar su influencia en la capacidad de extracción de características semánticas. [PENDIENTE: Detallar resultados]
- **Efecto de diferentes tipos de fusión en las skip connections:** Se compararon métodos como la concatenación simple frente a la adición o el uso de convoluciones 1x1 o 3x3 para refinar la fusión de mapas de características del encoder y decoder. [PENDIENTE: Detallar resultados y qué método de fusión funcionó mejor]
- **Influencia del tamaño de kernel:** Aunque RoiNet base usa kernels grandes (9x9), se podría haber experimentado con variaciones si formó parte del estudio. [PENDIENTE: Confirmar si se estudió y detallar]
- **Contribución de la función de pérdida SoftCLDice:** Se comparó el rendimiento del modelo entrenado con SoftCLDice frente a una función de pérdida más estándar como BCE+Dice. [PENDIENTE: Detallar resultados]

[PENDIENTE: Insertar tabla/gráficas con los resultados del estudio de ablación]

5.3 Evaluación de funciones de pérdida

Se evaluó el impacto de las diferentes funciones de pérdida especializadas:

- SoftCLDiceLoss mostró mejoras significativas en la conectividad vascular
- ConexLoss redujo notablemente las discontinuidades en vasos finos
- HaloCLDiceLoss proporcionó el mejor equilibrio entre precisión de bordes y conectividad

5.4 Análisis cualitativo

Además del análisis cuantitativo, se realizó una evaluación cualitativa de los resultados de segmentación, prestando especial atención a:

- Continuidad de estructuras vasculares
- Precisión en la detección de capilares finos
- Comportamiento en regiones patológicas
- Robustez ante variaciones en la calidad de imagen

5.5 Interpretación de Resultados y Discusión

Tal como se sugiere en las correcciones, es importante interpretar los resultados obtenidos a lo largo de los experimentos realizados en este TFG:

- **Rendimiento Comparativo de Arquitecturas:**
 - [PENDIENTE: Analizar en detalle qué arquitectura (FRNet vs. RoiNet modificada por el autor) segmenta mejor y bajo qué condiciones específicas, basándose en las métricas cuantitativas (Dice, AUC, Sensibilidad, Especificidad, etc.) y los resultados cualitativos. Por ejemplo, ¿una es mejor para vasos finos y otra para vasos gruesos? ¿Cómo se comportan ante diferentes patologías si el dataset lo permite?]
 - [PENDIENTE: Discutir las diferencias en eficiencia computacional si se midieron (e.g., tiempo de entrenamiento, tiempo de inferencia, uso de memoria GPU/CPU). ¿Cómo impactan estas diferencias la viabilidad de cada modelo?]
- **Características y Funcionamiento de FRNet y RoiNet Modificada:**
 - [PENDIENTE: Explicar las diferencias clave en el diseño conceptual (resolución completa vs. encoder-decoder), el tamaño del campo receptivo efectivo, la gestión de características multiescala, y cómo estas particularidades de FRNet y de la RoiNet modificada por el autor pueden explicar las diferencias observadas en el rendimiento. Por ejemplo, ¿cómo afecta la ausencia de downsampling en FRNet a la captura de contexto global en comparación con el bottleneck de RoiNet? ¿Qué papel juegan las skip connections refinadas en RoiNet?]
- **Ventajas y Limitaciones de los Elementos Arquitectónicos y de Entrenamiento Evaluados:**
 - [PENDIENTE: A partir del estudio de ablación y otras pruebas, discutir las ventajas y limitaciones observadas de distintos componentes específicos probados en RoiNet: por ejemplo, el impacto del número de bloques en el bottleneck, los diferentes métodos de fusión de skip connections, el efecto del tamaño de los kernels (si se varió), y la contribución de funciones de pérdida como SoftCLDice. ¿Qué se aprendió sobre la sensibilidad del modelo a estos cambios?]
- **Impacto de las Aportaciones Específicas del TFG:**
 - [PENDIENTE: Evaluar cómo las modificaciones, experimentos y desarrollos específicos realizados por el autor en la arquitectura RoiNet (configuraciones de entrenamiento, gestión de recursos en Cesga, pruebas con FIVES512, desarrollo de scripts, posible nueva función de pérdida) contribuyeron a los resultados finales y al conocimiento generado. ¿Qué problemas se resolvieron y qué nuevas perspectivas se abrieron?]
- **Gráficas y Diagramas:**
 - [PENDIENTE: Insertar aquí o referenciar las gráficas de validación y test que muestren la evolución del entrenamiento (pérdida, métricas) para las configuraciones más importantes.]

- [PENDIENTE: Insertar aquí o referenciar un diagrama que resuma las principales configuraciones arquitectónicas y de entrenamiento que se probaron, especialmente para RoiNet.]
- **Análisis de Casos Fallidos o Desafiantes:**
 - [PENDIENTE: Si es posible, mostrar y discutir ejemplos de imágenes donde los modelos (especialmente RoiNet modificado) fallan o tienen dificultades. ¿Qué características tienen estas imágenes? ¿Qué podría estar causando los errores? Esto puede ofrecer ideas para trabajo futuro.]

Esta sección debe servir para ir más allá de la simple presentación de números y figuras, ofreciendo una reflexión crítica sobre lo que significan los resultados en el contexto de los objetivos del TFG y del campo de la segmentación vascular.

Chapter 6

Conclusiones y Trabajo Futuro

6.1 Conclusiones principales

Este Trabajo Fin de Grado se ha centrado en el estudio, la adaptación experimental y la evaluación de arquitecturas de Redes Neuronales Convolucionales para la segmentación de vasos sanguíneos en imágenes de retina de alta resolución. Las principales conclusiones derivadas del trabajo realizado por el autor son:

1. **Análisis Comparativo de Enfoques:** Se han analizado dos enfoques arquitectónicos principales: FRNet (full-resolution) y RoiNet (encoder-decoder, también conocida como VesselView). Si bien FRNet presenta ventajas teóricas en la preservación de detalles al evitar el downsampling, la arquitectura RoiNet, sobre la cual se centraron las aportaciones de este TFG, demostró ser un marco flexible y potente para la experimentación y optimización. [PENDIENTE: Concretar con hallazgos específicos, e.g., "La RoiNet modificada alcanzó un Dice de X, superando a FRNet en Y bajo tales condiciones..."]
2. **Importancia de las Contribuciones Específicas sobre RoiNet:** Las modificaciones y experimentos llevados a cabo por el autor sobre RoiNet (incluyendo la optimización de configuraciones de entrenamiento, la gestión de experimentos en Cesga, la adaptación a FIVES512, la evaluación de funciones de pérdida como SoftCLDice, y los estudios de ablación sobre componentes clave) han sido fundamentales para entender el comportamiento del modelo y mejorar su rendimiento en la tarea de segmentación vascular. [PENDIENTE: Mencionar algún resultado destacado de estas contribuciones, e.g., "El estudio de ablación reveló que el método de fusión X en las skip connections mejoró el rendimiento en Z%"]
3. **Relevancia de las Funciones de Pérdida Especializadas:** La integración y evaluación de funciones de pérdida como SoftCLDice ha confirmado su utilidad para promover la conectividad vascular, un aspecto crítico que no siempre es bien capturado por funciones de pérdida más genéricas. [PENDIENTE: Cuantificar esta mejora si es posible]
4. **Desafíos de la Experimentación Práctica:** El trabajo ha puesto de manifiesto los desafíos inherentes a la experimentación con modelos de aprendizaje profundo en entornos de alto rendimiento y con datasets de alta resolución, tales como la gestión de recursos, los tiempos de entrenamiento y la necesidad de estrategias adaptativas (como el uso de FIVES512).
5. **Potencial de las Arquitecturas Estudiadas:** Tanto FRNet como, especialmente, la

versión de RoiNet trabajada en este TFG, han demostrado ser capaces de superar a los métodos tradicionales y ofrecer resultados competitivos para la segmentación vascular en imágenes de alta resolución, sentando una base para futuras mejoras.

6.2 Valoración de lo Aprendido

La realización de este TFG ha supuesto una valiosa experiencia de aprendizaje en múltiples facetas:

- **Profundización en Aprendizaje Profundo:** Se han consolidado y ampliado los conocimientos sobre arquitecturas de redes neuronales convolucionales (especialmente U-Net y sus variantes, y conceptos de redes full-resolution), funciones de pérdida, técnicas de regularización y optimización.
- **Habilidades de Experimentación Científica:** Se ha desarrollado la capacidad para diseñar experimentos, implementarlos, analizar resultados de forma crítica y extraer conclusiones válidas. Esto incluye la gestión de estudios de ablación y la comparativa rigurosa de modelos.
- **Competencias Técnicas en MLOps:** Se han adquirido habilidades prácticas en el manejo de frameworks de deep learning (PyTorch), la gestión de grandes datasets, el uso de plataformas de computación de alto rendimiento (Cesga, Slurm), y la refactorización y mantenimiento de código de investigación.
- **Resolución de Problemas:** Se ha fomentado la capacidad para identificar y solucionar problemas técnicos y metodológicos que surgen inevitablemente en un proyecto de investigación y desarrollo de esta naturaleza.
- **Comunicación Científica:** La elaboración de esta memoria y la preparación para su defensa han mejorado las habilidades para comunicar ideas complejas y resultados técnicos de forma clara y estructurada.
- [PENDIENTE: Añadir cualquier otra reflexión personal sobre el aprendizaje obtenido]

6.3 Trabajo futuro (sobre la evolución de RoiNet/SantosNet)

A partir del trabajo realizado en este TFG y las conclusiones obtenidas, se proponen las siguientes líneas de investigación y desarrollo futuro, enfocadas en la evolución y mejora de la arquitectura RoiNet/VesselView (denominada "SantosNet" en el contexto de las mejoras del autor):

1. **Desarrollo y Evaluación Exhaustiva de la Nueva Función de Pérdida:** Finalizar la implementación y realizar una evaluación rigurosa de la nueva función de pérdida cuyo desarrollo se inició en este TFG. Comparar su rendimiento con SoftCLDice y otras funciones de pérdida relevantes en diversos escenarios y métricas.
2. **Exploración Avanzada de Mecanismos de Atención:** Investigar e integrar módulos de atención más sofisticados (e.g., atención espacial, por canal, auto-atención o transformadores) dentro de la arquitectura RoiNet/SantosNet, con el objetivo de mejorar la capacidad de la red para enfocarse en estructuras vasculares relevantes y difíciles, especialmente los vasos finos.
3. **Optimización de Hiperparámetros y Arquitectura mediante Búsqueda Automatizada:** Emplear técnicas de búsqueda de arquitecturas neuronales (NAS) o de optimización de hiperparámetros (e.g., optimización bayesiana) para explorar de forma más sistemática

el espacio de configuraciones de RoiNet/SantosNet y encontrar variantes potencialmente superiores.

4. Mejora de la Generalización y Robustez:

- Evaluar la generalización del modelo entrenado en FIVES en otros datasets públicos de imágenes de retina (DRIVE, STARE, CHASE_DB1) para identificar posibles problemas de domain shift.
- Explorar técnicas de aumento de datos más avanzadas y estrategias de regularización para mejorar la robustez del modelo ante variaciones en la calidad de imagen, artefactos y diferentes tipos de patologías.

5. Aplicación a Tareas Clínicas Específicas:

- Adaptar el modelo para la cuantificación de biomarcadores vasculares relevantes (e.g., diámetro vascular, tortuosidad, densidad vascular).
- Investigar la extensión de la arquitectura para la segmentación multiclase (e.g., distinguir arterias de venas) o para la detección y segmentación simultánea de vasos y lesiones patológicas (e.g., microaneurismas, exudados).

6. Integración de Información Multimodal:

Si se dispone de otras modalidades de imagen (e.g., OCT, angio-OCT), explorar la fusión de información para mejorar la segmentación vascular.

7. Interpretación y Explicabilidad del Modelo:

Aplicar técnicas de interpretabilidad (e.g., mapas de activación, LIME, SHAP) para entender mejor qué características de la imagen está utilizando el modelo para tomar sus decisiones, lo que podría guiar futuras mejoras arquitectónicas o identificar sesgos.

8. [PENDIENTE: Añadir otras ideas específicas del autor para la evolución de RoiNet/Santos que no estén implementadas a fecha de entrega del TFG]

Chapter 7

Referencias bibliográficas

1. Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI).
2. He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
3. Staal, J., Abràmoff, M. D., Niemeijer, M., Viergever, M. A., van Ginneken, B. (2004). Ridge-based vessel segmentation in color images of the retina. IEEE Transactions on Medical Imaging.
4. Hoover, A., Kouznetsova, V., Goldbaum, M. (2000). Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response. IEEE Transactions on Medical Imaging.
5. Fraz, M. M., Remagnino, P., Hoppe, A., Uyyanonvara, B., Rudnicka, A. R., Owen, C. G., Barman, S. A. (2012). Blood vessel segmentation methodologies in retinal images – A survey. Computer Methods and Programs in Biomedicine.
6. Orlando, J. I., Fu, H., Breda, J. B., van Keer, K., Bathula, D. R., Diaz-Pinto, A., ... Bogunović, H. (2020). REFUGE Challenge: A unified framework for evaluating automated methods for glaucoma assessment from fundus photographs. Medical Image Analysis.
7. Maninis, K. K., Pont-Tuset, J., Arbeláez, P., Van Gool, L. (2016). Deep retinal image understanding. In Medical Image Computing and Computer-Assisted Intervention (MICCAI).
8. Milletari, F., Navab, N., Ahmadi, S. A. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. In Fourth International Conference on 3D Vision (3DV).
9. Gu, Z., Cheng, J., Fu, H., Zhou, K., Hao, H., Zhao, Y., ... Liu, J. (2019). CE-Net: Context Encoder Network for 2D Medical Image Segmentation. IEEE Transactions on Medical Imaging.
10. Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N., Liang, J. (2018). UNet++: A Nested U-Net Architecture for Medical Image Segmentation. In Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support.